When Local Evaluation Misleads Coding-Agent Development

Issue informativeness, harness fidelity, and trajectory adaptation under a fixed Gemma 4 inference budget

Tarun Kumar
The Pennsylvania State University

Abstract

Local evaluations must distinguish improvements in software repair from changes in task information, execution conditions, and sampling variability. We study these factors using a fixed Gemma 4 31B agent. On 44 bugs from 17 repositories, replacing full reports with their first lines reduces resolution from 37/44 (84.1%) to 26/44 (59.1%), with repositories, tests, configuration, and budgets held fixed: 14 tasks are lost and three gained (two-sided exact paired test, $p = 0.013$). A harness audit identifies four execution mismatches, including an import-path fault invalidating seven of 28 recorded solves in one repository. Prompt and budget interventions and three trajectory-trained LoRA adapters produce no consistent gain on a 25-task screen, although one adapter advances the median first edit from call 10 to seven. Across 13 configurations, the title-only screen correlates with public scores at Spearman's rho = 0.48 but fails its predefined cross-band criterion. We combine statement ablations, grading controls, paired comparisons, and runtime accounting to distinguish procedural adaptation from repair gains. Under the tested model and budget, issue content changed resolution more than any configuration change improved it.

1. Introduction

Coding agents must interpret an issue, navigate a repository, implement a change, and validate it before exhausting their calls or time. In the Gemma 4 Developer Agent Competition, participants configure a fixed base model and may train low-rank adapters within a prescribed execution stack and 12-hour aggregate runtime limit. Local evaluation is therefore central to selecting interventions.

Local resolution rates can be misleading. Informative reports may support repairs that terse descriptions do not, even with identical repositories and tests. Serving speed changes how much exploration fits within a time limit. Grader faults can produce apparent successes by testing an installed package instead of edited source. These effects complicate local-to-hidden comparisons, particularly when repeated submissions produce different scores.

Prior work studies underspecified software-engineering instructions and shows that interaction can help agents recover missing information (Vijayvargiya et al., 2026; Edwards and Schuster, 2026). We examine the complementary setting of autonomous repair without user clarification, where the agent must infer the intended behavior from the supplied statement and repository under a fixed budget.

We contribute a controlled statement experiment, a measured harness audit, calibration across 13 configurations, and prompt, budget, and trajectory-training experiments. These distinguish procedural changes, such as earlier editing or longer exploration, from repair gains. Statement ablations, reference- and empty-patch controls, and calibration against noisy scores apply beyond this competition to SWE-bench-style evaluations; the measured effect sizes remain specific to our experiments.

2. Experimental Setting and Evaluation Protocol

The agent receives a repository snapshot and issue statement, modifies source, and submits a patch for test-based grading. Approximately 120 hidden tasks must complete within 12 hours on four NVIDIA L4 GPUs (Markowitz et al., 2026).

Our baseline, bf, reproduces the public Budget-Fit V5 configuration. It uses Gemma 4 31B with QAT w4a16 quantization, served through the competition's patched vLLM 0.19.1 build. Its principal settings are:

Table 1. Baseline inference configuration.

Component	Baseline setting
Per-task limits	28 tool calls; eight minutes
Command timeout	240 seconds
Sampling	Temperature 0.2; top_p 0.95
Generation limits	4,096 thinking tokens; 8,192 output tokens
Context	32,768 tokens; past thoughts retained
Tools	File inspection, search, editing, shell execution, and patch submission

Later A100 experiments use ADK 1.36.1 and the updated harness, with compaction at 14,336 tokens. Earlier runs used preceding versions. Most ranking and adapter experiments use an A100; a Mac-based stack supports environment checks and trajectory generation. Matching time limits across hardware does not guarantee equal effective inference budgets.

bf leaves graph tools unused. External public ablations report 21/21 empty semantic searches (Thakur, 2026) and median gold-file rank 3.5 using BM25 over the file tree (BusyaPrime, 2026). These are environment-specific reports, not our graph-tool ablation.

We use complementary task collections, including tasks drawn from SWE-rebench (Badertdinov et al., 2025):

Table 2. Task collections and their experimental roles.

Collection	Size	Purpose
Public regression sets	89 sound tasks	Early diagnostics and regression checks
Unfamiliar repositories	27 tasks	Repository-familiarity and statement ablations
Primary title-only screen	25 tasks	Paired configuration comparisons
Naturally terse issues	30-34 tasks, depending on grader	Secondary evaluation
OOD-45	44 of 45 tasks sound locally	Controlled full-report versus first-line experiment
Batch-3 pool	134 locally sound tasks	Trajectory generation for later adapters

A task is retained only when its reference patch passes required tests and an empty patch fails. The title-only screen contains unfamiliar-repository tasks supported by both graders. Stronger-model attempts diagnose terse-task solvability; their patches are not training targets. OOD-45 and the primary screen are evaluation-only.

Candidates are paired with bf on identical tasks. Before each run, we record an advancement rule: a net gain of at least four of 25 tasks, with at most one strict loss, warrants a repeat and second-set evaluation. Differences within one task are operational ties, not equivalence findings.

We report paired gains and losses and two-sided exact tests on discordant outcomes. Leaderboard calibration evaluates published configurations under supplied budgets using rank correlation and a predefined cross-band criterion. Identical submissions characterize score variability; subsequent noise simulations are exploratory.

3. Issue Informativeness and Screen Calibration

We first test whether removing issue-report content changes resolution while holding the underlying tasks fixed. On OOD-45, bf receives either the dataset's rewritten user report or only its first line. The repository snapshots, tests, agent configuration, and nominal budgets remain unchanged. Forty-four tasks from 17 repositories pass our reference- and empty-patch controls.

Table 3. Paired statement ablations. Gains and losses are measured after truncation.

Paired experiment	Tasks	Full report solved	First line solved	Gained / lost
Unfamiliar repositories, v4ntfix	27	10 (37.0%)	4 (14.8%)	0 / 6
OOD-45, bf	44	37 (84.1%)	26 (59.1%)	3 / 14

Truncation reduces OOD-45 resolution by 25 percentage points: 14 tasks are lost and three gained (p = 0.013). A separate grading path reproduces all 44 full-report outcomes. The unfamiliar-repository experiment shows six losses and no gains (p = 0.031). Each comparison uses one stochastic attempt per task and condition. Figure 1 summarizes both ablations.

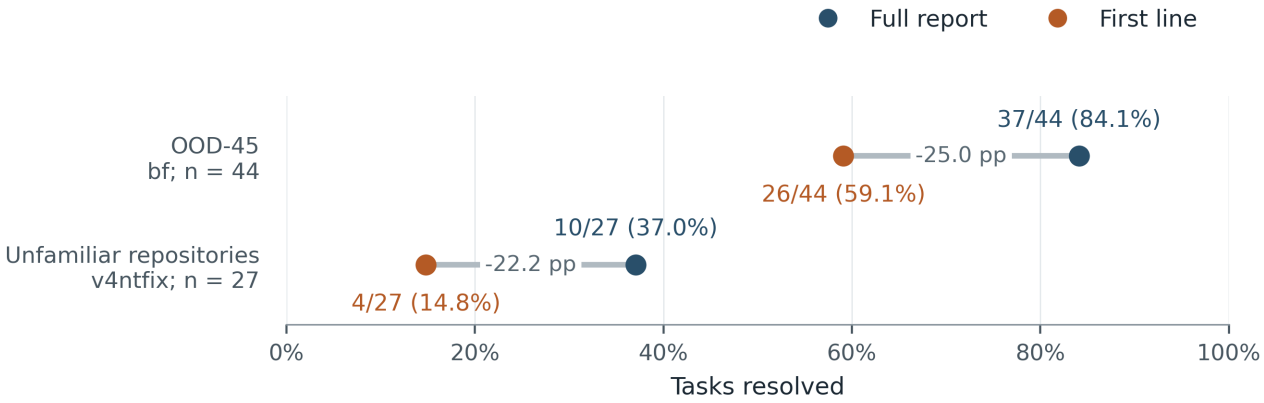


Figure 1. Full-report and first-line resolution on matched tasks. Connecting lines show within-collection changes; collections use different configurations. Counts are observed outcomes, without uncertainty intervals.

Training-data generation shows the same direction: on 111 Batch-3 tasks, full-report attempts solve 47 (42.3%), while first-line attempts solve 33 and 36 (29.7% and 32.4%). These are descriptive generation results, not held-out adapter evaluations. Truncation changes both length and available information.

Mechanism evidence is suggestive. An exploratory 30-task public-set ablation supplying exact gold-hunk locations raises solves only from 10 to 11; timing confounds limit interpretation. Title-only traces include relevant-file edits missing unstated sibling behavior or return values. In two of six unfamiliar-task losses, reports supply additional behavior or a failing input absent from the title, including two-decimal timestamps. These observed failures require behavioral specification beyond file localization; the evidence does not establish a single failure cause.

Task shape matters: bf solves 59.1% of first-line OOD-45, versus 32% on the primary screen; 40/44 OOD reference patches modify one file. Truncation does not reproduce a common difficulty level or establish the cause of lower hidden scores.

Across twelve configurations, only ten of the 25 title-only tasks are ever solved; each configuration solves three to eight of those same ten. This shared observed pool limits ranking separation and suggests task-dependent failures. It does not establish that the remaining tasks are unsolvable.

Across 13 configurations with comparable scoring epochs, including reproduced third-party bundles (Rodrigues, 2026b), Spearman's correlation is 0.48 (Figure 2). Our predefined rule labels 0.3-0.6 weak agreement and requires configurations scoring at most 0.10 publicly to rank below those scoring at least 0.13 in 75% of cross-band pairs.

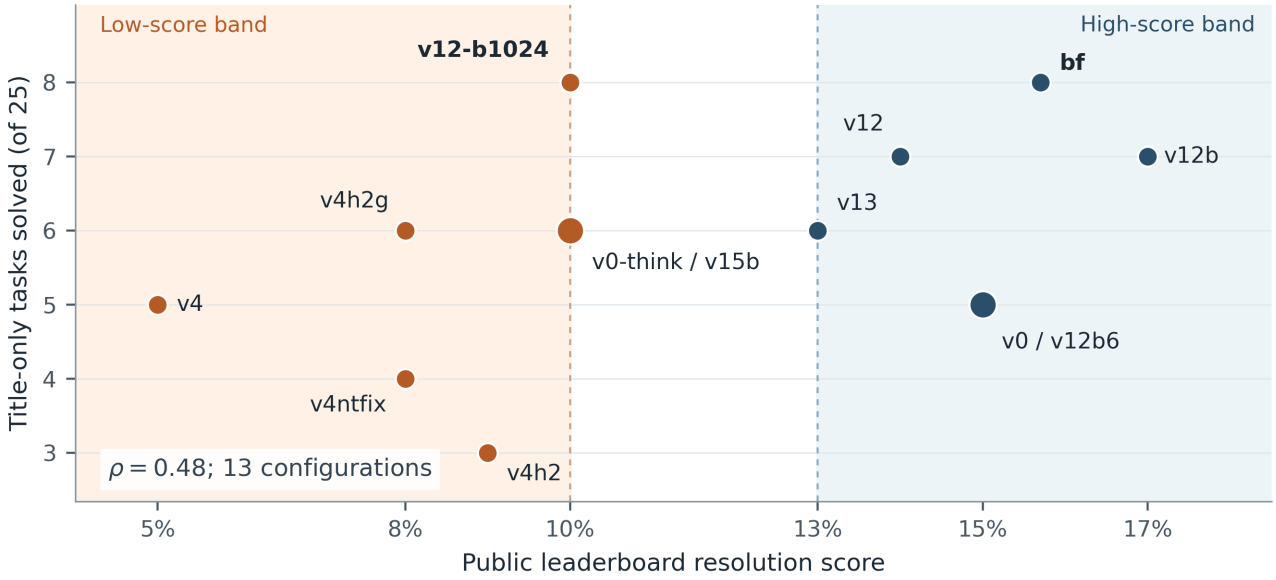


Figure 2. Thirteen configurations; two pairs overlap at identical coordinates. Shading marks predefined public-score bands. Labels expose inversions; local results do not meet the 75% cross-band criterion. Coordinates use reported mean public scores.

Of 42 cross-band pairs, 25 are correctly ordered, six tie, and 11 invert. Half credit for ties yields 66.7%, below the criterion. Several pipelines reach their time limits on most local tasks; faster A100 serving changes how many calls fit.

Identical bf uploads solve nine, ten, and nine of 58 public tasks. An exploratory binomial simulation gives median correlation 0.50 for an idealized informative screen (5th-95th percentiles: 0.10-0.79). Its assumptions neither establish a formal ceiling nor override the failed criterion.

4. Harness Fidelity

We audit whether local runs execute and grade the intended task under conditions comparable to the competition stack. Four mismatches emerge. Three affect grading validity; the fourth changes model inputs without producing a detectable resolution change in the comparison performed.

Table 4. Harness mismatches, observed effects, and corrections.

Mismatch	Observed effect	Correction
Rich tests import an installed release rather than edited source	Verdicts do not measure the submitted workspace changes	Rebuild the Docker image to import the workspace
Requests imports bypass its edited src-layout package	Seven of 28 previously recorded solves are false passes	Put the workspace source directory first on the import path
Starlette 1.x is incompatible with older FastAPI snapshots	Reference patches fail on 48 of 67 tasks; one task passes without a patch	Pin Starlette 0.52.1, recovering 35 tasks
ADK 2.x serializes tool results differently from the scorer's ADK 1.36.1	Input formatting differs; baseline reruns remain at seven to eight solves out of 25	Pin ADK 1.36.1 and inspect captured requests

Before accepting agent results, we grade reference and empty patches. An empty-patch pass invalidates a claimed repair; a reference-patch failure requires environment investigation. We repeat controls after dependency or import-path changes. Successful execution alone does not establish that tests exercised edited code.

ADK adds an identity line absent from traces, while serialization strips trailing thought whitespace. Accounting for these differences reconstructs prompt token counts on 18/18 captured requests. Complete training-turn fidelity remains unverified: compaction introduces summaries absent from traces.

5. Prompt, Budget, and Trajectory Adaptation

We test whether changes to instructions, reasoning budgets, and trajectory training improve resolution under the baseline's constraints.

Thinking affects both reliability and cost. Disabling it reduces local resolution from eight to four of 25 tasks. Retaining thinking while dropping prior thoughts produces four paired gains and four losses across 55 tasks, but approximately doubles completion tokens per call and increases timeouts. Its competition submission exceeds the aggregate 12-hour limit. An instruction requiring the first edit by call six moves the median only from call 10 to call nine: resolution falls from eight to six of 25, while call-cap hits increase from seven to 12. The intended budget policy is therefore only partially adopted.

For trajectory training, we use the QAT base loaded in four bits, rank-16 LoRA with alpha 16, a learning rate of 10^{-4}, two epochs, gradient accumulation of four, and a 16,384-token sequence limit. Loss applies to model-generated thoughts and actions; user messages and tool outputs are masked. Truncated trajectories are retained only when their final source edit survives. We remove duplicate runs and trajectories exceeding the served thought or tool-call limits. The first adapter predates the correction for ADK's omitted identity line; subsequent adapters include it.

Table 5. The 31B adapters on the same 25-task screen. Paired gains and losses are relative to bf.

Adapter	Training data	Sequences	Solved / baseline	Paired gains / losses
ad1	Passing trajectories from 28 public tasks	78	8 / 8	1 / 1
ad3A	Full-report trajectories from 55 Batch-3 tasks	59	7 / 8	1 / 2
ad3B	ad3A plus filtered first-line-condition trajectories	98	8 / 8	1 / 1

All comparisons use the same 25-task screen. Although ad3A advances the median first edit from call 10 to call seven, it produces no resolution gain. Ad3B instead extends execution: median calls rise from 20 to 25, and timeouts from two to seven. Earlier editing and longer exploration are thus insufficient to improve outcomes in these runs.

We distill the 31B agent into E4B using 160 teacher sequences, rank and alpha 128, and two epochs. On the same 25 tasks under both statement conditions, the adapter solves one title-only task against one, and five full-report tasks against six. The base typically stops after about nine calls and two minutes. Mean calls rise from 11.4 to 34.6 after adaptation, with four timeouts against none: procedure changes without repair gains. A further evaluation batch remains unrun after the job reaches its wall-clock limit.

6. Related Work

SWE-bench establishes repository-level issue resolution as an evaluation task (Jimenez et al., 2024). SWE-agent studies how an agent-computer interface supports navigation, editing, and testing (Yang et al., 2024), while Agentless uses a prescribed localization, repair, and validation pipeline (Xia et al., 2024). Agentless also identifies insufficient or misleading issue descriptions, placing task quality alongside scaffold design as an evaluation concern. Our study examines these concerns through controlled statement ablations and measured grading faults.

Ambig-SWE evaluates agents under underspecified instructions and studies whether clarification recovers missing information (Vijayvargiya et al., 2026). Ask or Assume? develops agents that detect underspecification and seek clarification during execution (Edwards and Schuster, 2026). We examine autonomous repair without user interaction, measuring the effect of removing report content under fixed budgets and testing whether a terse local screen agrees with hidden-task scores.

SWE-Gym provides executable training environments and demonstrates improvements from training agents and verifiers (Pan et al., 2025). SWE-smith scales task generation across repositories and trains a capable software-engineering model (Yang et al., 2025). These successes motivate trajectory adaptation, but differ from our small-data, low-rank experiments. Our null results bound the interventions tested here and highlight the need to measure repair outcomes alongside learned execution patterns.

Rodrigues (2026a), What a Local Screen Could Not Tell Us, reports that full-statement screening did not rank hidden scores. We add a controlled statement experiment, calibration using his published bundles, and an exploratory noise analysis.

7. Limitations and Conclusion

The study covers one model family and small sets. Repeated screen use risks selection bias; 25 tasks cannot reliably resolve modest gains or establish equivalence. Exact paired tests do not account for shared repositories.

First-line truncation is a proxy, not a reconstruction of unavailable hidden statements. OOD-45's predominantly single-file fixes limit generalization across task complexity. The noise model is exploratory and assumption-dependent.

Hardware affects effective budgets; traces omit compaction summaries. Adaptation findings apply to the tested datasets, ranks, and recipes. E4B evaluation ends after two statement conditions.

Removing report content reduces resolution by 25 percentage points on 44 tasks, while harness auditing removes false passes. Prompting and trajectory imitation alter execution without consistent repair gains. Paired comparisons, grading controls, and runtime accounting distinguish capability from measurement effects. Under the tested model and budget, issue content changed resolution more than any configuration change improved it.

Data and code availability. Public-release links are pending packaging of the terse sets, OOD-45 first-line variant, sound-task lists, per-task outcomes, grader fixes, and renderer. No public release is claimed here.

References

Badertdinov, I., et al. (2025). SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411. https://arxiv.org/abs/2505.20411

BusyaPrime. (2026). 119 of 129 sound: the Gemma 4 grader, rebuilt [Kaggle notebook]. https://www.kaggle.com/code/busyaprime/119-of-129-sound-the-gemma-4-grader-rebuilt

Edwards, N., and Schuster, S. (2026). Ask or Assume? Uncertainty-Aware Clarification-Seeking in Coding Agents. arXiv:2603.26233. https://arxiv.org/abs/2603.26233

Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR. https://arxiv.org/abs/2310.06770

Markowitz, E., et al. (2026). Google - The Gemma 4 Developer Agent Competition. Kaggle. https://www.kaggle.com/competitions/gemma-4-developer-agent

Pan, J., et al. (2025). Training Software Engineering Agents and Verifiers with SWE-Gym. ICML, PMLR 267, 47717-47737. https://proceedings.mlr.press/v267/pan25g.html

Rodrigues, C. (2026a). What a Local Screen Could Not Tell Us. Zenodo. https://doi.org/10.5281/zenodo.23592038

Rodrigues, C. (2026b). gemma-swe-agent [Source repository; commit ac45389]. GitHub. https://github.com/rodriguescarson/gemma-swe-agent

Thakur, A. (2026). Agent ablations [Kaggle discussion]. https://www.kaggle.com/competitions/gemma-4-developer-agent/discussion/746250

Vijayvargiya, S., et al. (2026). Ambig-SWE: Interactive Agents to Overcome Underspecificity in Software Engineering. ICLR. https://arxiv.org/abs/2502.13069

Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489. https://arxiv.org/abs/2407.01489

Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS. https://arxiv.org/abs/2405.15793

Yang, J., et al. (2025). SWE-smith: Scaling Data for Software Engineering Agents. arXiv:2504.21798. https://arxiv.org/abs/2504.21798